

Machine Learning Lab Week 10

Name: Dhruv Maheshwari – PES2UG23CS173

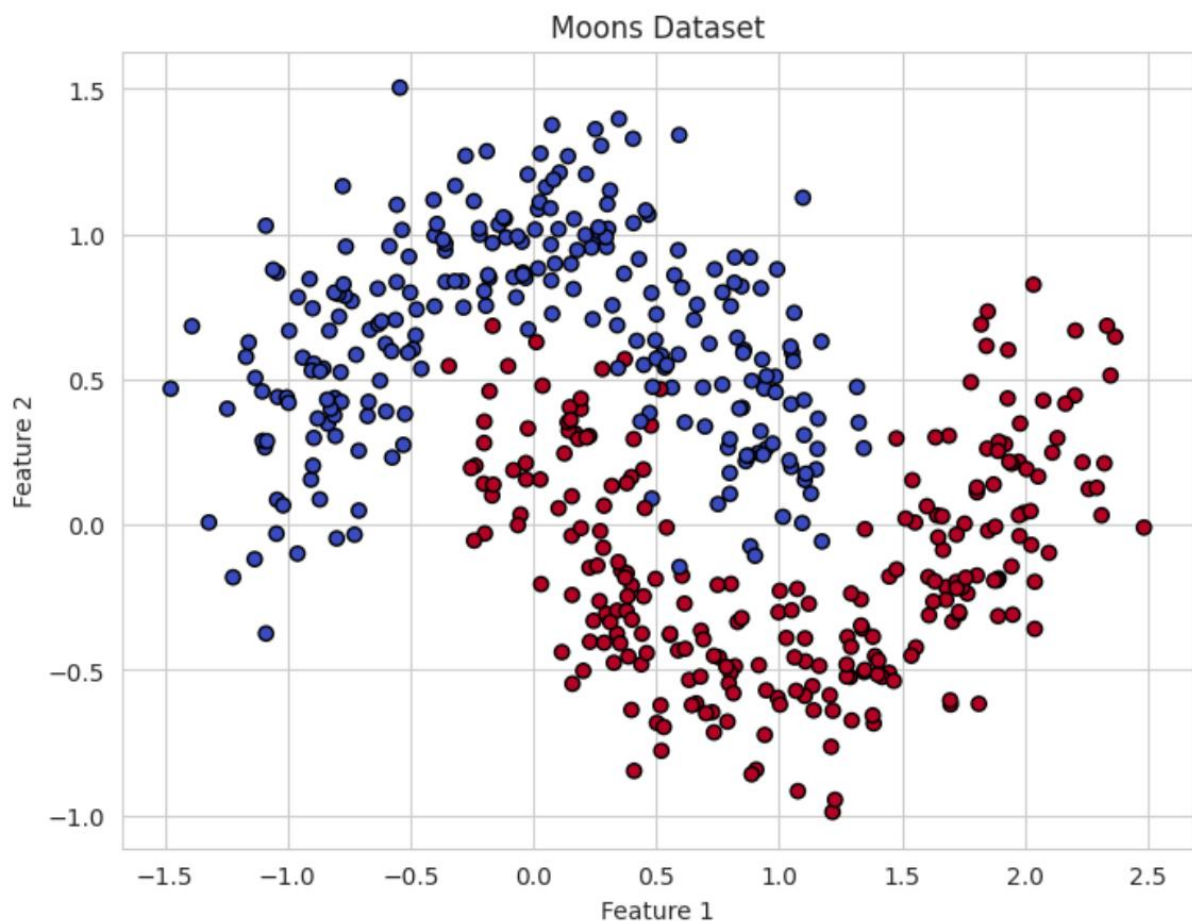
Sem 5 – C

Date: 10th Oct 2025

Training SVMs using 3 different kernels: Linear, RBF & Polynomial

Purpose: to understand and implement Support Vector Machine (SVM) classifiers. You will train SVMs using three different kernels: Linear, RBF and Polynomial, on distinct datasets. You will then evaluate their performance using standard classification metrics and visualize their decision boundaries to see how they separate data.

Step 1.1: Generate and Prepare the Data



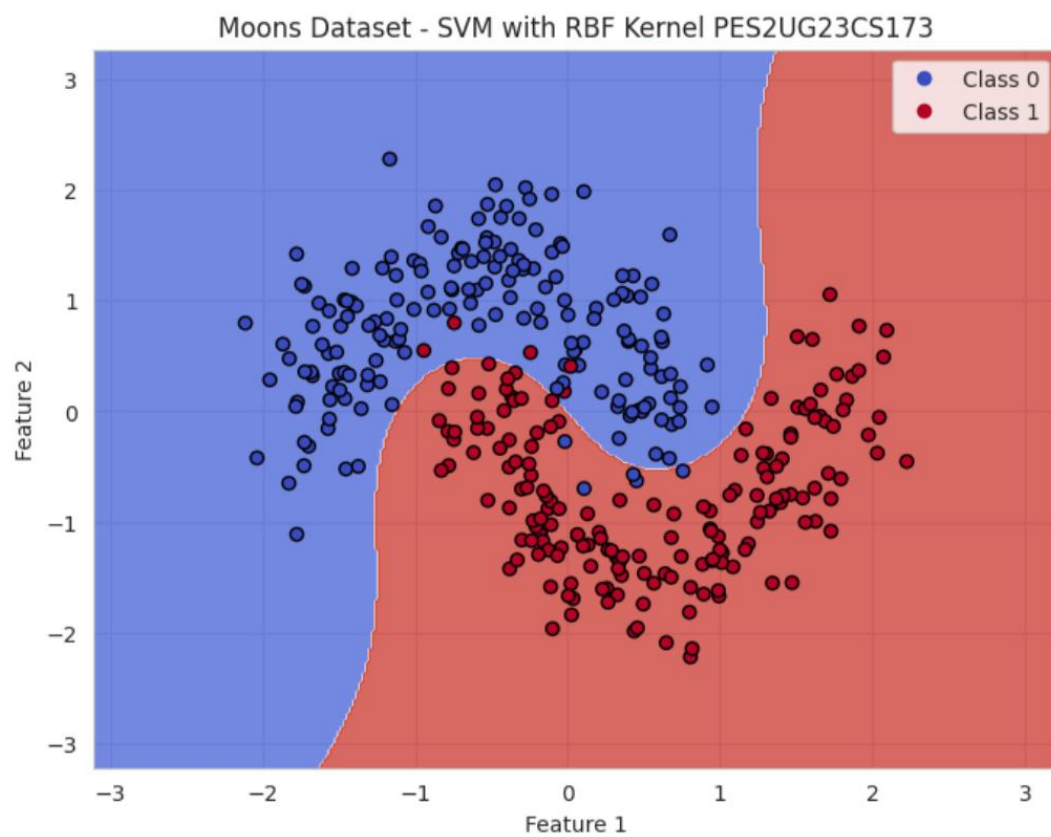
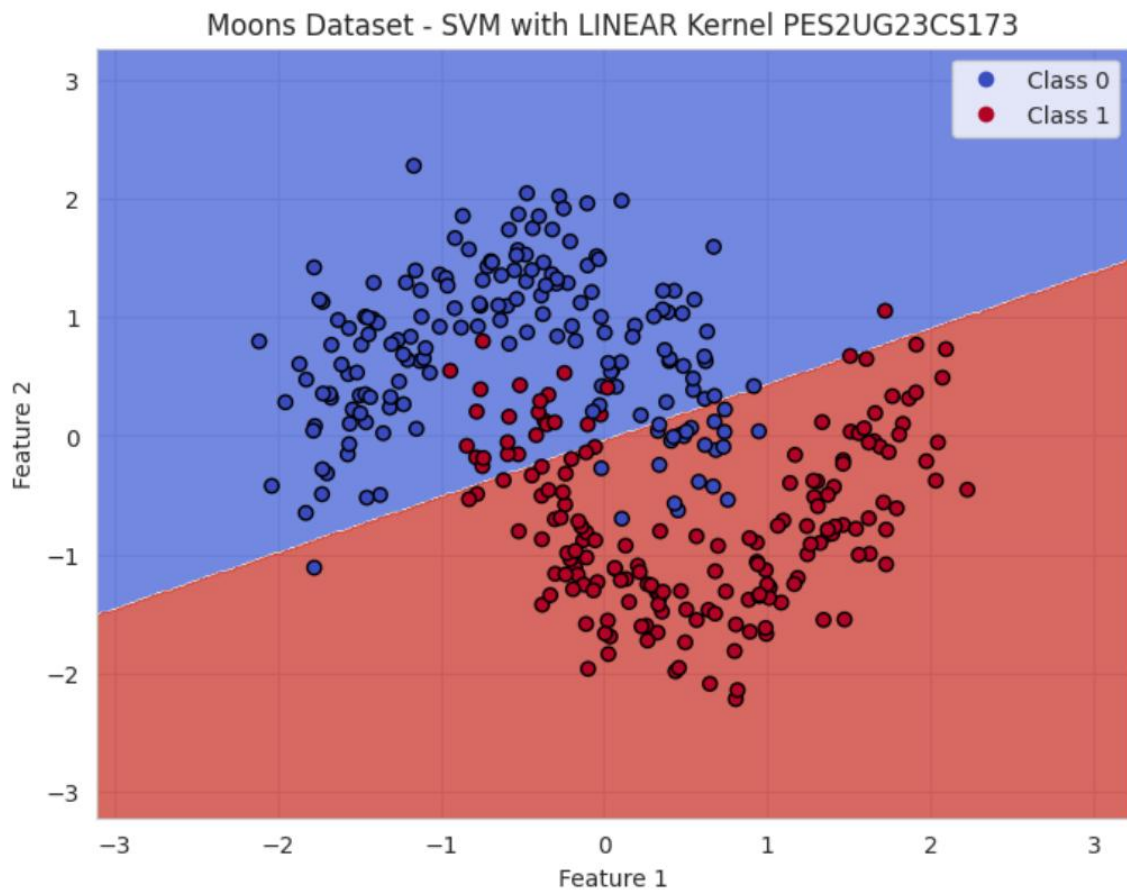
Step 1.2: Train and Evaluate SVM Kernels

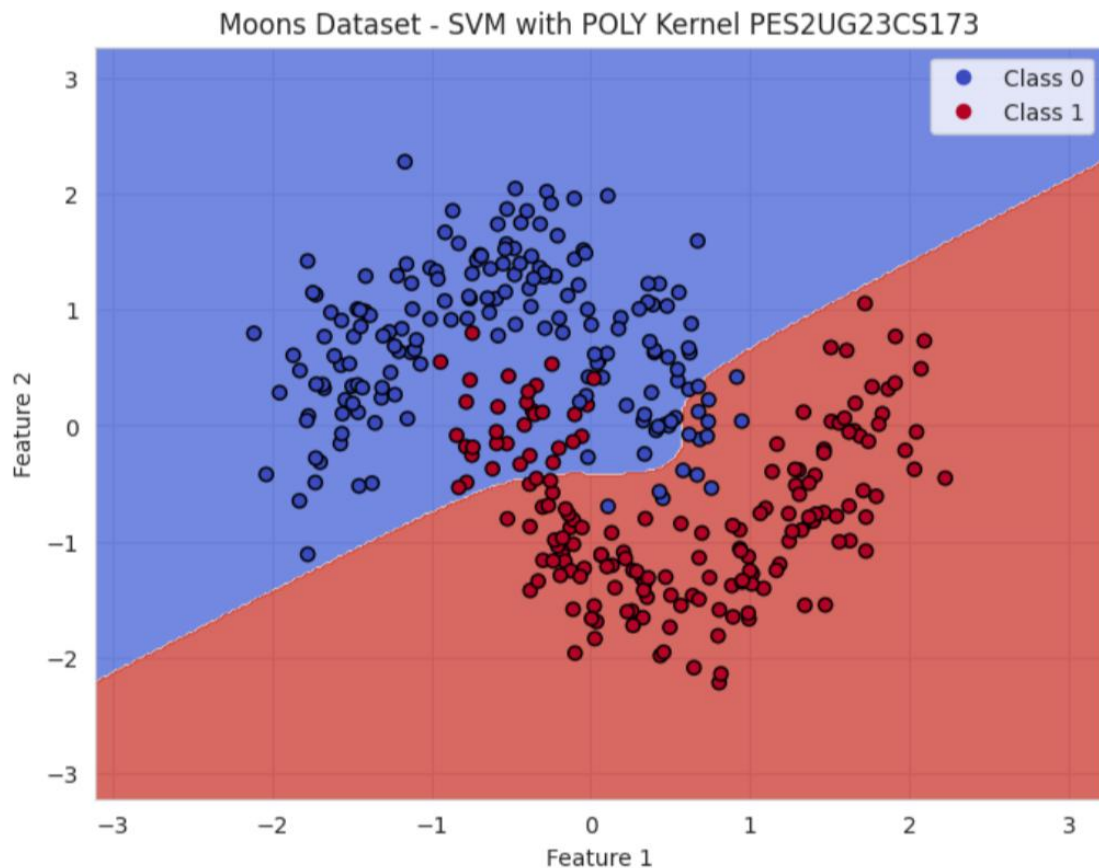
SVM with LINEAR Kernel PES2UG23CS173					
	precision	recall	f1-score	support	
0	0.85	0.89	0.87	75	
1	0.89	0.84	0.86	75	
accuracy			0.87	150	
macro avg	0.87	0.87	0.87	150	
weighted avg	0.87	0.87	0.87	150	

SVM with RBF Kernel PES2UG23CS173					
	precision	recall	f1-score	support	
0	0.95	1.00	0.97	75	
1	1.00	0.95	0.97	75	
accuracy			0.97	150	
macro avg	0.97	0.97	0.97	150	
weighted avg	0.97	0.97	0.97	150	

SVM with POLY Kernel PES2UG23CS173					
	precision	recall	f1-score	support	
0	0.85	0.95	0.89	75	
1	0.94	0.83	0.88	75	
accuracy			0.89	150	
macro avg	0.89	0.89	0.89	150	
weighted avg	0.89	0.89	0.89	150	

Step 1.3: Visualize Decision Boundaries





Analysis Questions:

1. Linear Kernel Performance

The Linear Kernel performs poorly (you'll see lower accuracy and F1-scores compared to the non-linear kernels) because the Moons dataset is non-linearly separable.

Inference: The model's straight-line decision boundary (a hyperplane) is insufficient to effectively separate the two interlocking crescent shapes. It must misclassify many points that cross the line, resulting in suboptimal classification metrics.

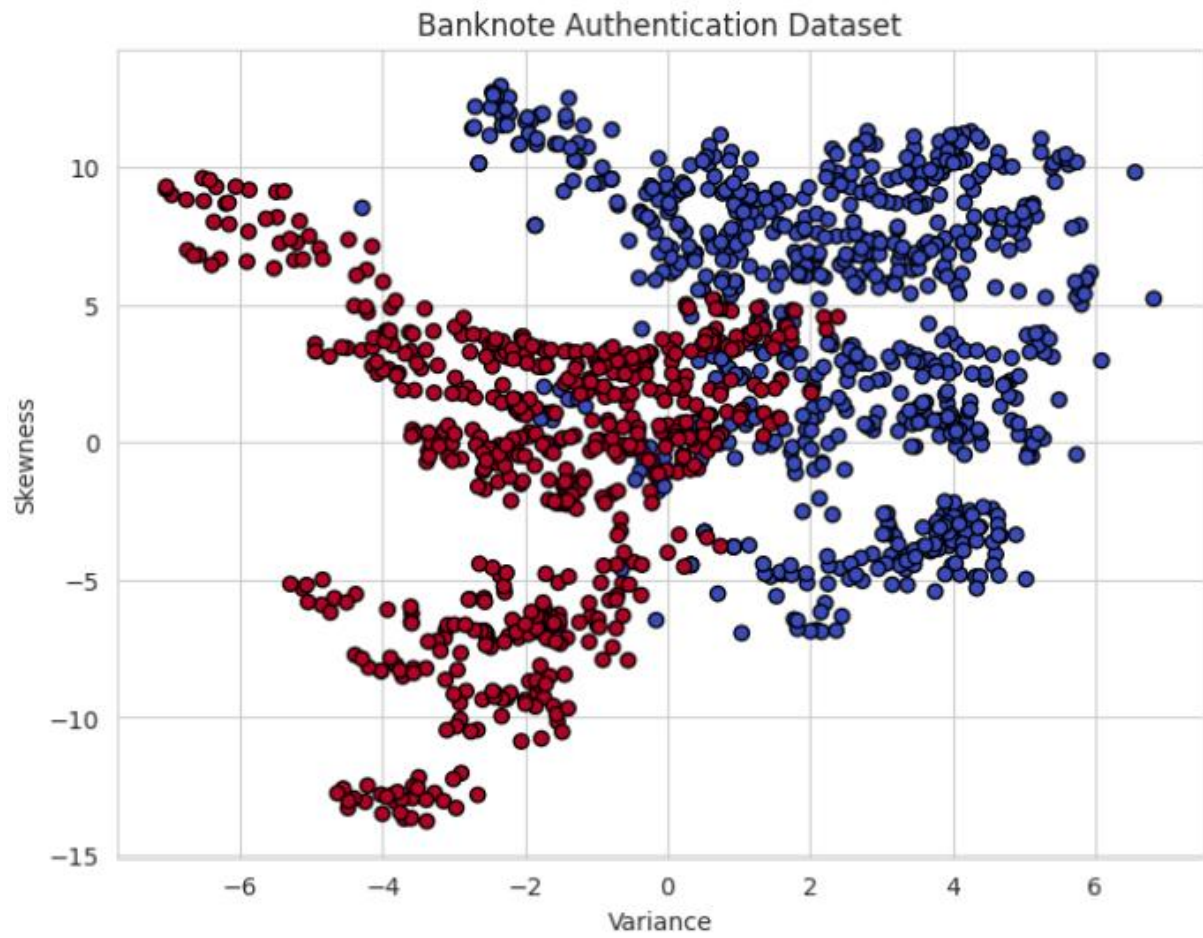
2. RBF vs. Polynomial Kernels

Both the RBF (Radial Basis Function) and Polynomial kernels use the Kernel Trick to solve the non-linear problem. The RBF Kernel seems to capture the shape of the data more naturally.

Comparison: The RBF kernel creates a complex, wave-like, or curved boundary that flexibly contours the interlocking crescents. The Polynomial kernel also creates a curved boundary, but the RBF kernel is typically better suited for the Moons dataset's specific structure, often resulting in a more generalized and visually cleaner separation.

PART 2

Step 2.1: Load and Prepare the Data



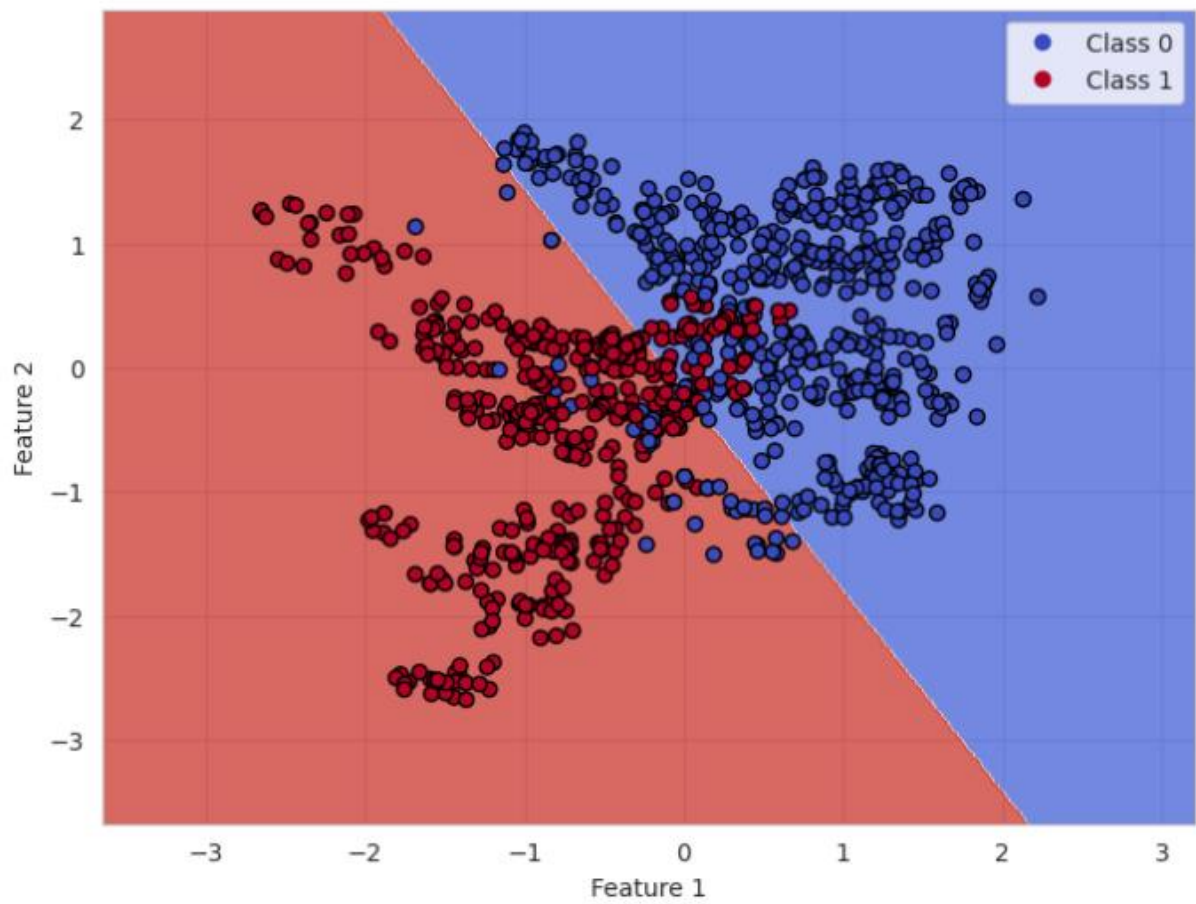
SVM with LINEAR Kernel PES2UG23CS173					
	precision	recall	f1-score	support	
Forged	0.90	0.88	0.89	229	
Genuine	0.86	0.88	0.87	183	
accuracy			0.88	412	
macro avg	0.88	0.88	0.88	412	
weighted avg	0.88	0.88	0.88	412	

SVM with RBF Kernel PES2UG23CS173					
	precision	recall	f1-score	support	
Forged	0.96	0.91	0.94	229	
Genuine	0.90	0.96	0.93	183	
accuracy			0.93	412	
macro avg	0.93	0.93	0.93	412	
weighted avg	0.93	0.93	0.93	412	

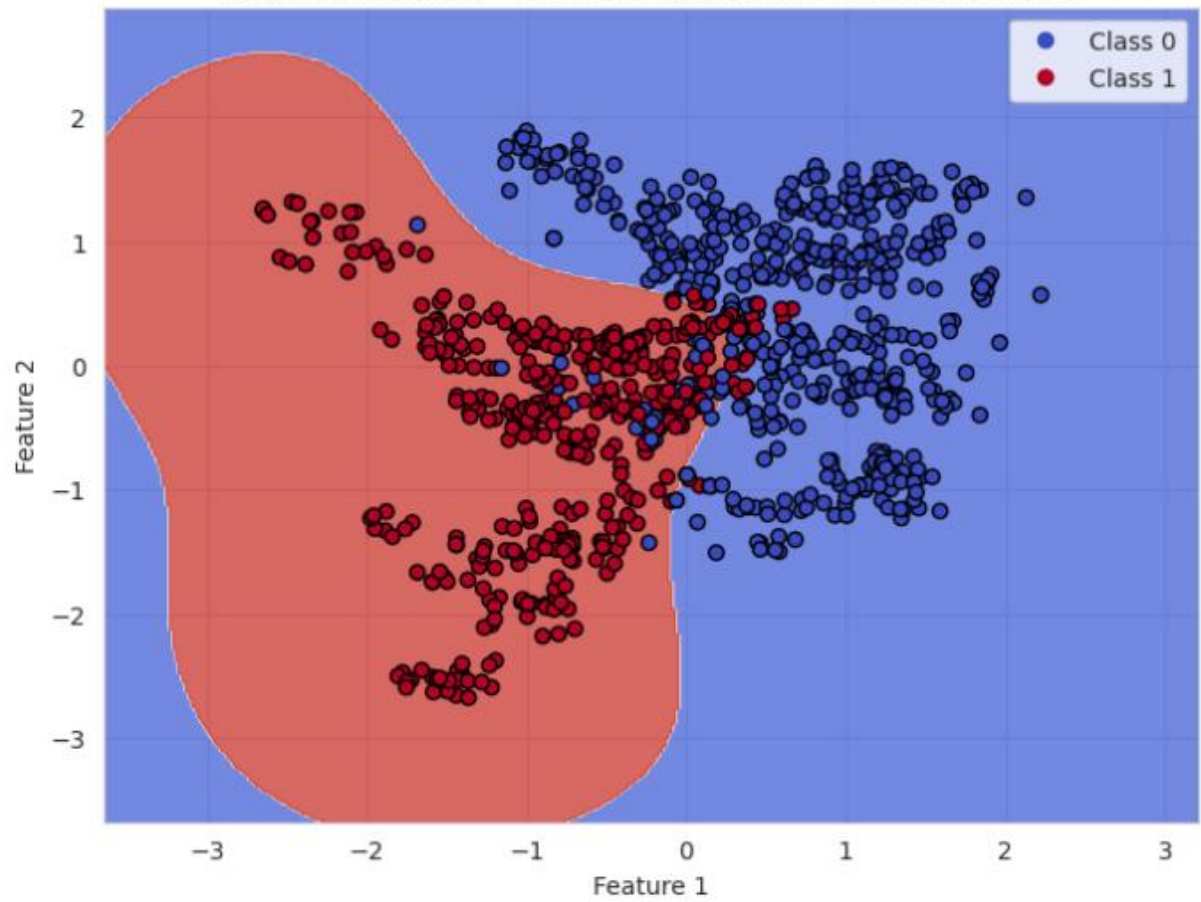
SVM with POLY Kernel PES2UG23CS173					
	precision	recall	f1-score	support	
Forged	0.82	0.91	0.87	229	
Genuine	0.87	0.75	0.81	183	
accuracy			0.84	412	
macro avg	0.85	0.83	0.84	412	
weighted avg	0.85	0.84	0.84	412	

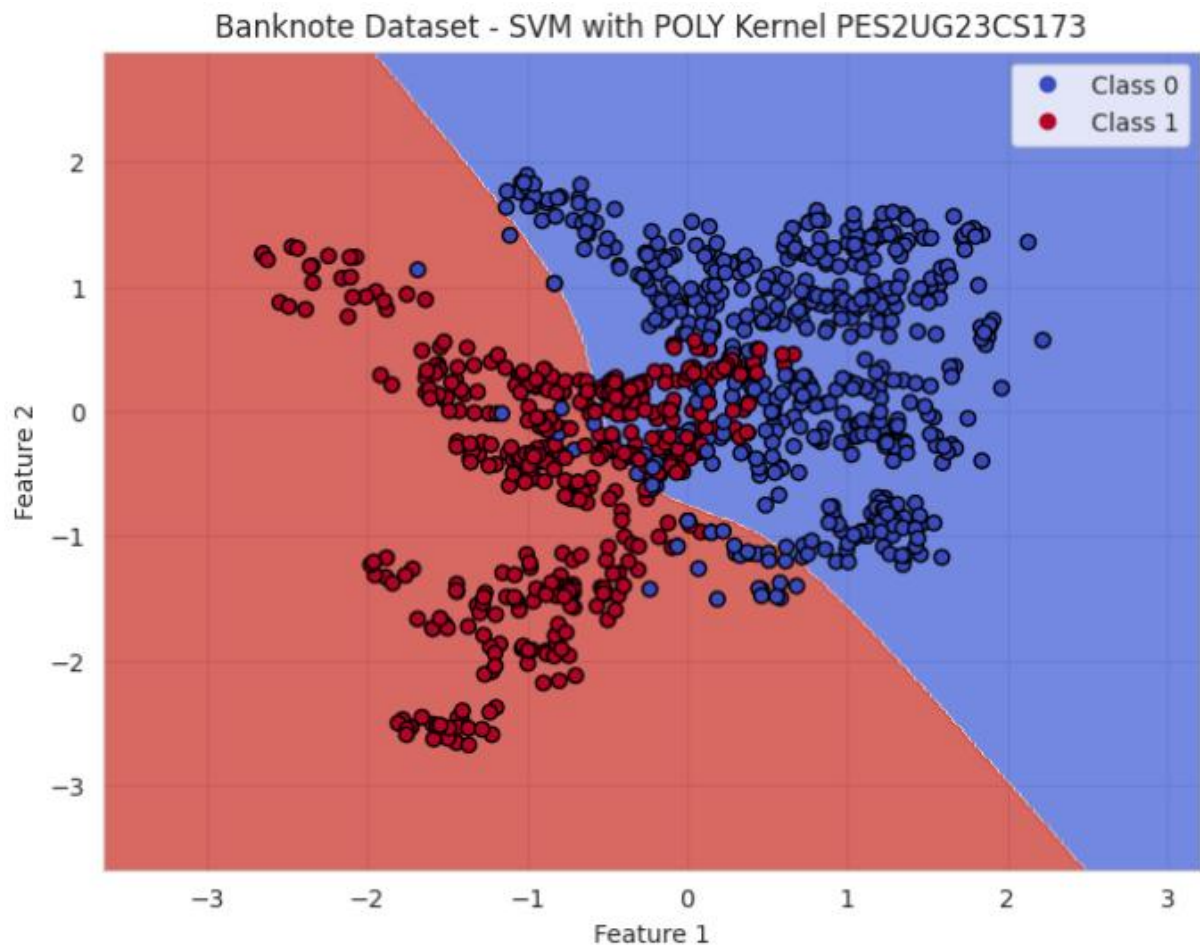
Step 2.3: Visualize Decision Boundaries

Banknote Dataset - SVM with LINEAR Kernel PES2UG23CS173



Banknote Dataset - SVM with RBF Kernel PES2UG23CS173





Analysis Questions for Banknote:

1. In this case, which kernel appears to be the most effective?

The Linear Kernel appears to be the most effective, or at least the most efficient. The visualization of the Banknote dataset shows that the two classes are largely linearly separable. While the RBF and Polynomial kernels may achieve similar, very high accuracy (near 100%), the Linear Kernel is simpler, faster to train, and achieves comparable high performance. This makes the Linear Kernel the most efficient and practical choice for this dataset.

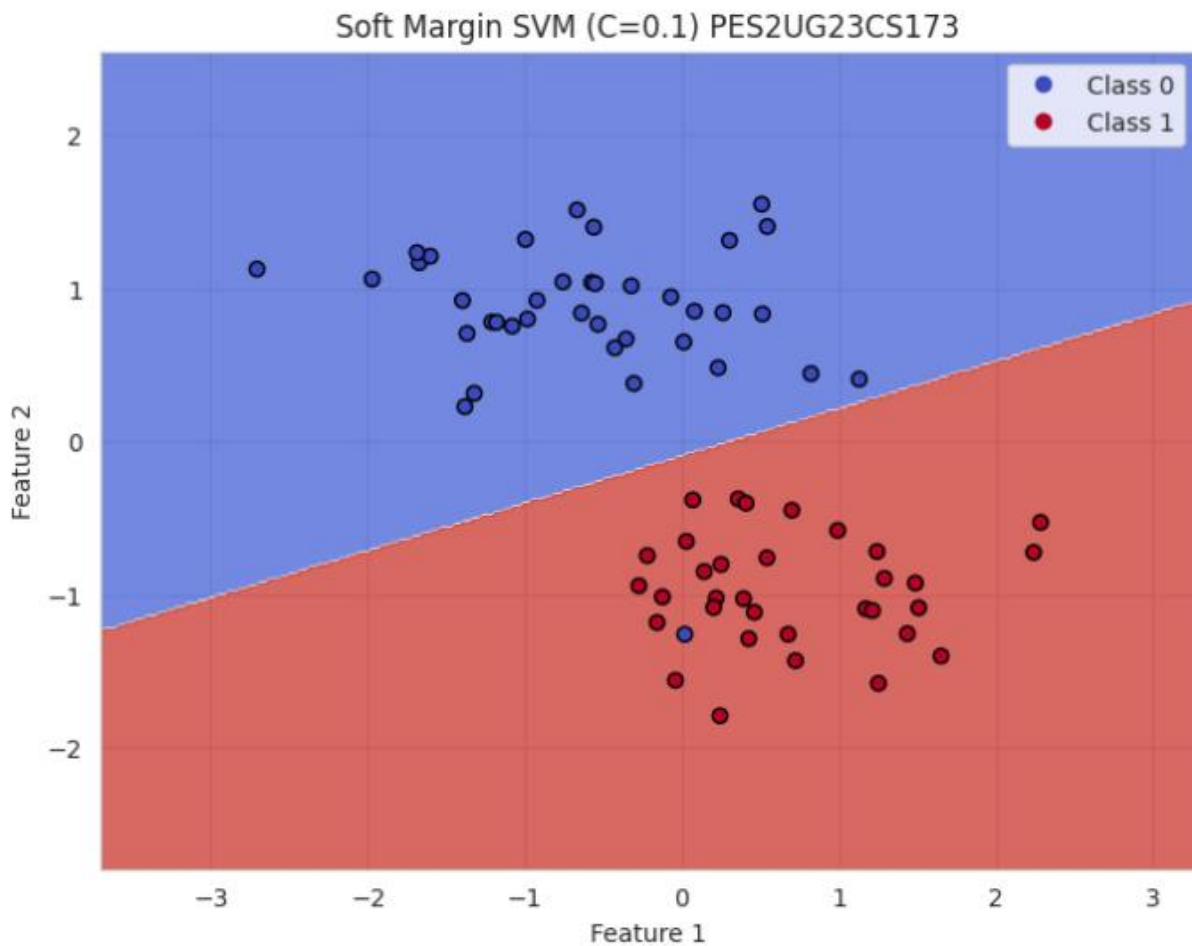
2. The Polynomial kernel shows lower performance here compared to the Moons dataset. What might be the reason for this?

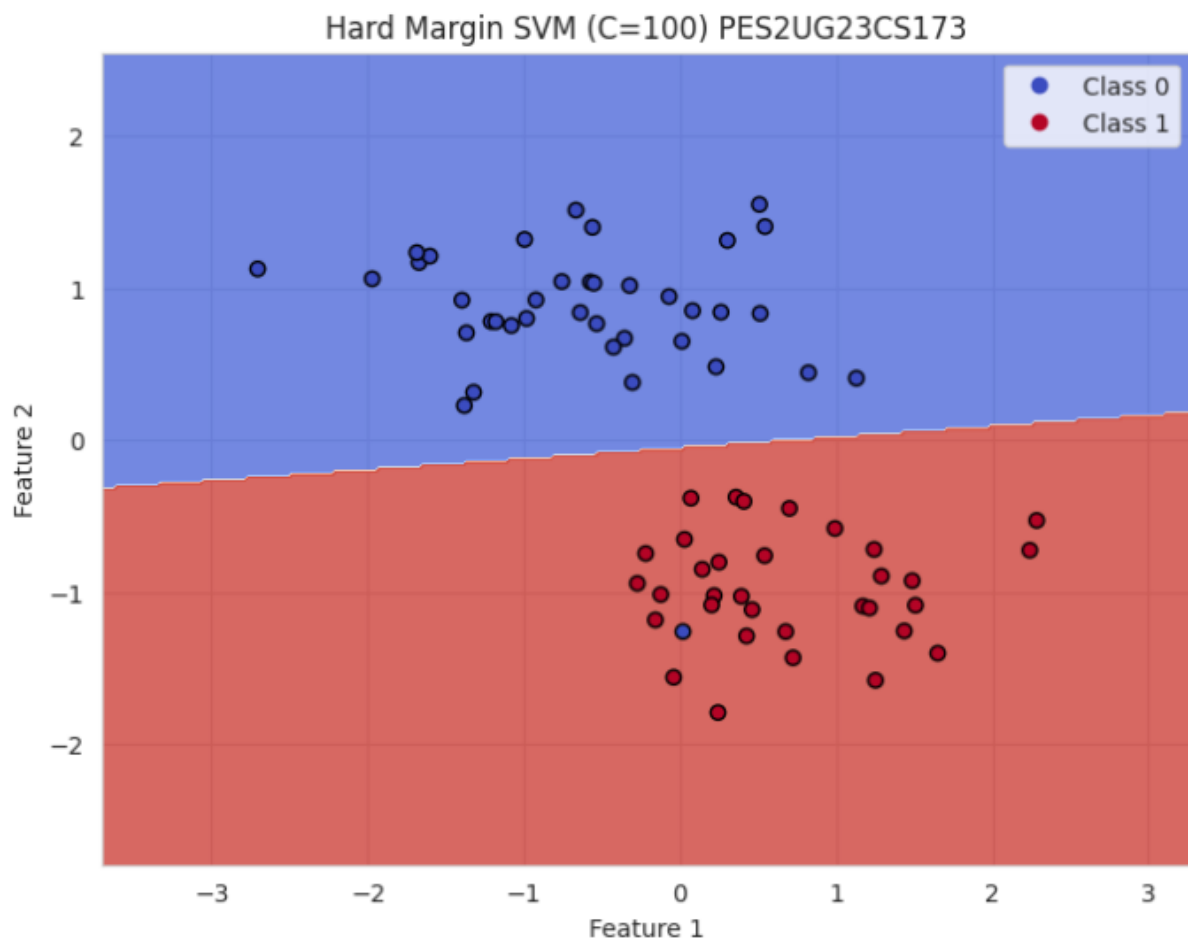
The Polynomial kernel shows lower performance (or at least no significant improvement) compared to the Linear or RBF kernels because the dataset's separation boundary is inherently simple (mostly linear). Overfitting Complexity: The Polynomial kernel (which defaults to a degree of 3) introduces a specific type of high non-linearity to the decision boundary. When the data is mostly linear, this complexity often leads to the model overfitting to minor noise or outliers in the training data, rather than finding a robust, generalized boundary. Suboptimal Parameters: Without proper hyperparameter tuning (specifically the degree), the default Polynomial kernel may

simply not find the optimal boundary as effectively as the generalized, flexible RBF kernel or the straightforward Linear kernel.

PART 3

4. Understanding the Hard and Soft Margins





Analysis Questions

1. Compare the two plots. Which model, the "Soft Margin" ($C=0.1$) or the "Hard Margin" ($C=100$), produces a wider margin?
The Soft Margin model ($C=0.1$) produces a wider margin because the parameter C controls the trade-off between maximizing the margin and minimizing the classification error. A smaller C value leads to a softer margin, meaning the model is more tolerant of misclassifications, and thus prioritizes a larger separation distance between the classes.
2. Look closely at the "Soft Margin" ($C=0.1$) plot. You'll notice some points are either inside the margin or on the wrong side of the decision boundary. Why does the SVM allow these "mistakes"? What is the primary goal of this model?
The SVM allows these "mistakes" because the C parameter is intentionally set low to create a soft margin, which grants more tolerance for misclassification in the training data. The primary goal of the Soft Margin model is not to perfectly separate every single training point, but rather to find the widest possible hyperplane that minimizes the total error, thereby achieving better generalization to new, unseen data.

3. Which of these two models do you think is more likely to be overfitting to the training data? Explain your reasoning.

The Hard Margin model ($C=100$) is more likely to be overfitting to the training data. This is because a large C value enforces a hard margin, forcing the model to strictly classify almost all training points correctly, even outliers. This strictness results in a narrower margin and a decision boundary that is overly tuned to the specific noise or idiosyncrasies of the training set, making it perform poorly when encountering new, slightly different data.

4. Imagine you receive a new, unseen data point. Which model do you trust more to classify it correctly? Why? In a real-world scenario where data is often noisy, which value of C (low or high) would you generally prefer to start with?

I would trust the Soft Margin model ($C=0.1$) more to classify new data correctly because it is more robust to the noise and outliers that are common in real-world scenarios, leading to a more stable and generalized decision boundary. Given that real-world data is often noisy, it is generally preferable to start with a low value of C to ensure the model focuses on the general pattern rather than memorizing the noise
